

13. Header Linked List – Time Complexity and Space Complexity –Part 2

1. Delete from Beginning:

- **Time Complexity:**

→ **$Best = Worst = Average = O(1).$**

- **Space Complexity:**

→ **No Parametric Input hence, Parametric Input Space:**

$Input Space = O(1).$

$Auxiliary Space = O(1).$

Total Space Complexity: $Input Space + Auxiliary = O(1) + O(1) = O(1).$

→ **Full Data Definition :**

$Input Space[Considering the whole list] = O(n).$

$\left[\begin{array}{l} \text{Deleting one node , total nodes reduces to } n - 1 = \\ O(n - 1) = O(n). \end{array} \right]$

$Auxiliary Space = O(1).$

$\left[\begin{array}{l} \text{This is the extra space or temporary memory} \\ \text{used by the algorithm to complete} \\ \text{its task, excluding the space taken by} \\ \text{the input data.} \end{array} \right]$

Total Space Complexity: $Input Space + Auxiliary = O(n) + O(1) = O(1).$

2. Delete At End:

- *Time Complexity:*

$$\rightarrow \text{Best} = \Omega(1). \left[\text{When List is Empty} \right]$$

$$\rightarrow \text{Worst Case} = O(n) = \text{Average Case} = \Theta(n). \\ \left[\begin{array}{l} \text{When List is not empty and there is numerous} \\ \text{Nodes in the list. Note: True Average Case doesn't} \\ \text{exist here.} \end{array} \right]$$

- *Space Complexity:*

$\rightarrow$ Parametric Input Space:

There is no parameter present hence:

$$\begin{aligned} \text{Input Space} &= O(1). \\ \text{Auxiliary Space} &= O(1). \end{aligned}$$

$$\text{Total Space Complexity: Input Space} + \text{Auxiliary} = O(1) + O(1) = O(1).$$

$\rightarrow$ Full Data Definition:

Here Input Space includes `n` no. of elements at `n` no. of nodes and last node gets deleted i.e. $n - 1$, then:

$$\begin{aligned} \text{Input Space} &= n - 1 = O(n - 1) = O(n). \\ \text{Auxiliary Space} &= O(1). \end{aligned}$$

$$\text{Total Space Complexity: Input Space} + \text{Auxiliary} = O(n) + O(1) = O(n).$$

3. Delete From Position:

- *Time Complexity:*

$$\rightarrow \text{Best} = \Omega(1). \left[\begin{array}{c} \text{When List is Empty} \\ \text{Or} \\ \text{Pos} < 1 \text{ or } \text{Pos} > \text{Head} \rightarrow \text{Data} \end{array} \right]$$

$\rightarrow \text{Worst Case} = O(n)$ [When `pos` is not found.]

$\rightarrow$ Average Case:

$\rightarrow$ If the position is 1, the loop's inner statement runs 0 time.

$\rightarrow$ If the position is 2, the loop's inner statement runs 1 time.

$\rightarrow$ If the position is 3, the loop's inner statement runs 2 time.

.

.

$\rightarrow$ If the position is (N), the loop's inner statement runs (N – 1) times.

As when $i = N < N$ becomes false, hence no execution of loop's inner statements.

$$\text{Hence, } \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}.$$

Now, as we take all the possibilities then, we take the best case also i.e.

$O(1)$, when position = 1. Considering `T(P)` is time complexity of position `P`.

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ P - 1 & , \text{when } P = 2, 3, 4, \dots, N. \end{cases}$$

Now, If there is `N` no. of nodes there we can delete data at N positions.

For position 1, the probability of deletion is $\frac{1}{N}$.

For position 2, the probability of deletion is $\frac{1}{N}$.

.
.

For position N, the probability of deletion is $\frac{1}{N}$.

Hence probability of deletion will always remain $\frac{1}{N}$.

$\therefore$ Average Time Complexity is : probability of deletion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N} \times 1 + \frac{1}{N} \times 2 + \dots + \frac{1}{N} \times N - 1 \right) + \frac{1}{N} \times 1$$

$$= \frac{1}{N} \times \{(1 + 2 + 3 + \dots + N - 1) + 1\}$$

$$= \frac{1}{N} \times \left\{ \sum_{i=1}^{N-1} i + 1 \right\}$$

$$= \frac{1}{N} \times \left(\frac{N(N-1)}{2} + 1 \right)$$

$$= \frac{N-1}{2} + \frac{1}{N}$$

Applying Big Theta in the equation:

$$= \Theta\left(\frac{N-1}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

Now, taking out the dominant term of numerator and denominator:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant '\frac{1}{2}', we get:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ terms , largest growth rate is the result:

→ $\Theta(N)$ grows linearly with ' N ' .

→ $\Theta\left(\frac{1}{N}\right)$ decreases to a constant, i.e. when N grows larger, $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0, \text{ where } c \text{ is constant}\right].$$

As $\Theta(N)$'s growth rate is higher, therefore, average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

*Average Case Time Complexity[Most Simplest Alternative Approach]:
(Simplest approach)*

For position 1, the probability of deletion is $\frac{1}{N}$.

For position 2, the probability of deletion is $\frac{1}{N}$.

.

.

For position N , the probability of deletion is $\frac{1}{N}$.

Hence probability of deletion will always remain $\frac{1}{N}$, for position 1 to N .

Average case complexity : Probability of deletion $\times$ $\left(\begin{array}{c} \text{Summation} \\ \text{of number of} \\ \text{Position.} \end{array}\right)$

$$= \frac{1}{N} \times \left(\sum_{i=1}^N i \right)$$

$$= \frac{1}{N} \times \left(\frac{N(N+1)}{2} \right)$$

$$= \frac{N+1}{2}$$

$$\text{or, } \frac{N}{2} + \frac{1}{2}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

Now removing the constant '\(\frac{1}{2}\)', we get:

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

Hence, Average Time Complexity is : $\Theta\left(\frac{N+1}{2}\right)$ or $\Theta(N)$.

- **Space Complexity:**

→ Parametric Input Space:

There is a parameter `int pos` present hence:

Input Space = $O(1)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: Input Space + Auxiliary = $O(1) + O(1) = O(1)$.

→ Full Data Definition:

Here Input Space includes `n` no. of elements at `n` no. of nodes and if a node gets deleted i. e. $n - 1$, then:

Input Space = $n - 1 = O(n - 1) = O(n)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: Input Space + Auxiliary = $O(n) + O(1) = O(n)$.

4. Destroy List:

- *Time Complexity:*

$$\rightarrow \text{Best} = \Omega(1). \left[\text{When List is Empty} \right]$$

$$\rightarrow \text{Worst Case} = O(n) = \text{Average Case} = \Theta(n). \\ \left[\begin{array}{l} \text{When List is not empty and there is numerous} \\ \text{Nodes in the list. Note: True Average Case doesn't} \\ \text{exist here.} \end{array} \right]$$

- *Space Complexity:*

$\rightarrow$ Parametric Input Space:

There is no parameter present hence:

$$\begin{aligned} \text{Input Space} &= O(1). \\ \text{Auxiliary Space} &= O(1). \end{aligned}$$

$$\text{Total Space Complexity: Input Space} + \text{Auxiliary} = O(1) + O(1) = O(1).$$

$\rightarrow$ Full Data Definition:

Before Deletion:

$$\begin{aligned} \text{List `n` nodes} &\rightarrow O(n). \left[\text{Input Space} \right] \\ \text{Auxiliary Space} &\rightarrow O(1). \end{aligned}$$

$$\begin{aligned} \text{Total Space Complexity: Input Space} + \text{Auxiliary} &= O(n) + O(1) \\ &= O(n). \end{aligned}$$

After Deletion:

- ***List becomes empty $\rightarrow O(1)$. [Input Space]***
- ***Auxiliary Space $\rightarrow O(1)$.***

Total Space Complexity: Input Space + Auxiliary = $O(1) + O(1) = O(1)$.

5. Free Header:

- ***Time Complexity:***

$\rightarrow \text{Best} = \text{Worst} = \text{Average} = O(1)$.

- ***Space Complexity:***

$\rightarrow$ For both, Parametric Input Space and Full Data Definition:

***Input Space = $O(1)$.
Auxiliary Space = $O(1)$.***

[As, DestroyList() function already executed.]

Total Space Complexity: Input Space + Auxiliary = $O(1) + O(1) = O(1)$.
